

ASSIGNMENT-13.3

NAME:P.Varshitha

BATCH NO.:0

ROLLNO:2403A510G8

TASK-1: Remove Repetition

PROMPT:

```
63  
64 It uses a series of if/elif statements to calculate the area of different shapes.  
65 I would like you to improve it by removing this conditional logic.  
66 Please propose a solution that is more modular, scalable, and follows good object-oriented design principles.  
67  
68  
69  
70
```

CODE:

```
13.3.py x
13.3.py > ...
1 import math
2
3 class Shape:
4     """A base class for all shapes."""
5     def area(self):
6         raise NotImplementedError("Subclasses must implement this method")
7
8 class Rectangle(Shape):
9     def __init__(self, width, height):
10         self.width = width
11         self.height = height
12
13     def area(self):
14         return self.width * self.height
15
16 class Square(Shape):
17     def __init__(self, side):
18         self.side = side
19
20     def area(self):
21         return self.side * self.side
22
23 class Circle(Shape):
24     def __init__(self, radius):
25         self.radius = radius
26
27     def area(self):
28         return math.pi * self.radius ** 2
29
30 # --- How to use it ---
31
32 # Create instances of shapes
33 my_rectangle = Rectangle(10, 5)
34 my_square = Square(7)
35 my_circle = Circle(3)
36
37 # Calculate areas without needing to know the shape type
38 print(f"Rectangle area: {my_rectangle.area()}") # Output: Rectangle area: 50
39 print(f"Square area: {my_square.area()}") # Output: Square area: 49
40 print(f"Circle area: {my_circle.area()}") # Output: Circle area: 28.274333882308138
41
42 # To add a new shape, just create a new class:
43 class Triangle(Shape):
44     def __init__(self, base, height):
45         self.base = base
46         self.height = height
47
48     def area(self):
49         return 0.5 * self.base * self.height
50
51 my_triangle = Triangle(10, 5)
52 print(f"Triangle area: {my_triangle.area()}") # Output: Triangle area: 25.0
53
```

OUTPUT:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\Praneeth Cheekati\OneDrive\Desktop\ai> & "C:/Users/Praneeth Cheekati/AppData/Local/Microsoft/WindowsApps/python3.11.exe" "c:/Users/Praneeth Cheekati/OneDrive/Desktop/ai/13.3.py"
Rectangle area: 50
Square area: 49
Circle area: 28.274333882308138
Triangle area: 25.0
PS C:\Users\Praneeth Cheekati\OneDrive\Desktop\ai> & "C:/Users/Praneeth Cheekati/AppData/Local/Microsoft/WindowsApps/python3.11.exe" "c:/Users/Praneeth Cheekati/OneDrive/Desktop/ai/13.3.py"
```

OBSERVATION:

```
57 Please propose a solution that is more modular, scalable, and follows good object-oriented design principles.
58
59
60 The original calculate_area function uses a long if/elif chain to determine which area calculation to perform.
61 Violates the Open/Closed Principle:
62 To add a new shape, you must modify the existing function.
63 A well-designed system should be open for extension but closed for modification.
64 Poor Scalability:
65 As more shapes are added, the function becomes increasingly long and complex, making it harder to read and maintain.
66 Mixing Concerns:
67 The function mixes the logic for dispatching (figuring out the shape) with the calculation logic for all shapes.
68
69
70
```

TASK-2: Error Handling in Legacy Code PROMPT:

13.3.py 9+ ●

13.3-1.py 9+ ●

13.3-1.py

```
1 I have the following legacy Python function.  
2 It's supposed to read a file, but it has a bug (.close() is called on a string) and no error handling. '  
3 Please refactor this code to be robust and safe. '  
4 It should use modern Python practices to handle potential '  
5 fileNotFoundError exceptions and guarantee that the file is always closed properly after being opened."
```

CODE:

```
13.3.py 9+ • 13.3-1.py X
13.3-1.py > ...
1 def read_file_safely(filename: str) -> str | None:
2     """
3     Reads data from a file with robust error handling.
4
5     This function uses a 'with' statement to ensure the file is automatically
6     closed and a 'try...except' block to gracefully handle exceptions.
7
8     Args:
9     |     filename (str): The path to the file to be read.
10
11     Returns:
12     |     str | None: The content of the file as a string, or None if an error occurs.
13     """
14     try:
15         # The 'with' statement ensures f.close() is called automatically
16         # after the block is exited, even if errors occur.
17         with open(filename, "r", encoding="utf-8") as f:
18             data = f.read()
19         return data
20     except FileNotFoundError:
21         # Gracefully handle the case where the file does not exist.
22         print(f"Error: The file '{filename}' was not found.")
23         return None
24     except IOError as e:
25         # Handle other potential input/output errors (e.g., permissions).
26         print(f"Error reading file '{filename}': {e}")
27         return None
28
29 # --- How to use it ---
30
31 # 1. Create a dummy file to test the success case
32 with open("my_data.txt", "w") as f:
33     f.write("Hello, world!")
34
35 # 2. Test with a file that exists
36 content = read_file_safely("my_data.txt")
37 if content:
```

```

32 with open("my_data.txt", "w") as f:
33     f.write("Hello, world!")
34
35 # 2. Test with a file that exists
36 content = read_file_safely("my_data.txt")
37 if content:
38     print(f"Successfully read content: '{content}'")
39 # Expected Output: Successfully read content: 'Hello, world!'
40
41 print("-" * 20)
42
43 # 3. Test with a file that does NOT exist
44 non_existent_content = read_file_safely("file_that_does_not_exist.txt")
45 if non_existent_content is None:
46     print("Function correctly handled the missing file.")
47 # Expected Output:
48 # Error: The file 'file_that_does_not_exist.txt' was not found.
49 # Function correctly handled the missing file.
50

```

OUTPUT:

```

PS C:\Users\Praneeth Cheekati\OneDrive\Desktop\ai> & "C:/Users/Praneeth Cheekati/AppData/Local/Microsoft/WindowsApps/python3.11.exe" "c:/Users/Praneeth Cheekati/OneDrive/Desktop/ai/13.3-1.py"
Successfully read content: 'Hello, world!'
-----
Error: The file 'file_that_does_not_exist.txt' was not found.
Function correctly handled the missing file.
PS C:\Users\Praneeth Cheekati\OneDrive\Desktop\ai>

```

OBSERVATION:

Observation

The refactored code is a significant improvement over the legacy version, making it robust, safe, and easier to understand.

- Bug Fix:** The original code would have crashed with an `AttributeError` because it incorrectly tried to call `.close()` on a string. The refactored code correctly reads the data into the `data` variable and lets the `with` statement handle closing the file.
- Resource Safety with `with open()`:** The `with` statement is the standard for working with resources like files in Python. It guarantees that the file is properly closed the moment the code exits the `with` block, regardless of whether it completes successfully or an error is raised. This prevents resource leaks.
- Error Handling with `try...except`:** The `try...except` block prevents the program from crashing if the file doesn't exist. Instead of an unhandled `FileNotFoundError`, it now catches the exception, prints a user-friendly error message, and returns `None`. This allows the code that calls the function to handle the failure gracefully.
- Modern Practices:** The use of type hints (`filename: str, -> str | None`) and specifying the file encoding (`encoding="utf-8"`) makes the code clearer, more predictable, and less prone to bugs when used by others.

TASK-3: Complex Refactoring PROMPT:

```
I have this legacy Python Student class that needs refactoring.  
  
The variable names are cryptic (n, a, m1), it lacks documentation, and the way it handles marks is not scalable.  
Please refactor this class to improve its readability and modularity.  
I'd like to see cleaner variable names, proper docstrings,  
' and a more flexible way to store and calculate the total marks, perhaps using a list."
```

CODE:

```
13.3.py 9+ • 13.3-1.py from typing import List Untitled-1 •
1  from typing import List
2
3  class Student:
4      """
5      Represents a student with their name, age, and a list of marks.
6      """
7
8      def __init__(self, name: str, age: int, marks: List[float]):
9          """
10         Initializes a Student object.
11
12         Args:
13             name (str): The full name of the student.
14             age (int): The age of the student.
15             marks (List[float]): A list of the student's marks.
16         """
17         self.name = name
18         self.age = age
19         self.marks = marks
20
21     def __str__(self) -> str:
22         """
23         Returns a user-friendly string representation of the student's details.
24         This method is automatically called by print().
25         """
26         return f"Name: {self.name}, Age: {self.age}"
27
28     def calculate_total_marks(self) -> float:
29         """
30         Calculates the sum of all marks for the student.
31
32         Returns:
33             float: The total of the student's marks.
34         """
35         return sum(self.marks)
36
37     def calculate_average_mark(self) -> float:
```



```

3 class Student:
28     def calculate_total_marks(self) -> float:
32         Returns:
34             """
35             return sum(self.marks)
36
37     def calculate_average_mark(self) -> float:
38         """
39         Calculates the average of all marks.
40
41         Returns:
42             float: The average mark, or 0.0 if there are no marks.
43         """
44         if not self.marks:
45             return 0.0
46         return sum(self.marks) / len(self.marks)
47
48 # --- How to use the refactored class ---
49
50 # Create a student instance with a list of marks
51 student1 = Student(name="John Doe", age=21, marks=[85, 90, 78, 92])
52
53 # Print details using the __str__ method
54 print(student1)
55 # Expected Output: Name: John Doe, Age: 21
56
57 # Calculate and print the total and average marks
58 total = student1.calculate_total_marks()
59 average = student1.calculate_average_mark()
60
61 print(f"Total Marks: {total}") # Expected Output: Total Marks: 345
62 print(f"Average Mark: {average:.2f}") # Expected Output: Average Mark: 86.25
63

```

OUTPUT:

```

PS C:\Users\Praneeth Cheekati\OneDrive\Desktop\ai> & "C:/Users/Praneeth Cheekati/AppData/Local/Microsoft/WindowsApps/python3.11.exe" "c:/Users/Praneeth Cheekati/OneDrive/Desktop/ai/13.3-2.PY"
Name: John Doe, Age: 21
Total Marks: 345
Average Mark: 86.25
PS C:\Users\Praneeth Cheekati\OneDrive\Desktop\ai>

```

OBSERVATION:

The original `Student` class is functional but suffers from several issues common in older or hastily written code:

- **Poor Naming:** Single-letter variable names like `n`, `a`, and `m1` are cryptic and make the code difficult to understand without context.
- **Lack of Documentation:** Without docstrings, a new developer has to read the implementation to understand what the class and its methods do.
- **Inflexible Design:** The marks (`m1`, `m2`, `m3`) are hardcoded as separate attributes. If a student had two marks or five marks, the class structure would need to be changed. This violates the Open/Closed principle.
- **Rigid Methods:** The `total` method is not scalable and would need to be updated every time the number of marks changes. The `details` method prints directly to the console, which mixes logic with presentation and makes the method less reusable.

The goal of the refactoring is to address these issues by applying modern Python best practices.

TASK-4: Inefficient Loop Refactoring PROMPT:


```
nums = [1,2,3,4,5,6,7,8,9,10] squares = []  
for i in nums: squares.append(i * i).give  
me the refacted code
```

CODE:

```
13-3-4.py > ...  
1  # List of numbers from 1 to 10  
2  nums = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]  
3  
4  squares = [i * i for i in nums]  
5  
6  # Printing the original list of numbers  
7  print("Original numbers:", nums)  
8  
9  # Printing the list of squares  
10 print("Squares of the numbers:", squares)  
11
```

OUTPUT:

```
> & C:/Users/DELL/AppData/Local/anaconda3/python.exe "c:/Users/DELL/  
Desktop/vs code/.vscode/13-3-4.py"  
Original numbers: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]  
Squares of the numbers: [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]  
PS C:\Users\DELL\Desktop\vs code\.vscode>
```

OBSERVATION:

The refactored code replaces the multi-line `for` loop with a single, highly expressive **list comprehension**. This is the preferred method in modern Python for several reasons:

1. **Conciseness and Readability:** The list comprehension `[i * i for i in nums]` is a single line that clearly and succinctly describes its purpose: "create a new list containing `i * i` for each `i` in `nums`". For Python developers, this is more readable than the original three-line loop.
2. **Performance:** List comprehensions are generally faster than explicit `for` loops that use `.append()`. This is because the looping and appending logic is handled by highly optimized C code at the interpreter level, avoiding the overhead of repeated `.append()` method calls in Python.
3. **Declarative Style:** The list comprehension is more *declarative* (it describes *what* you want) rather than *imperative* (it describes *how* to do it step-by-step). This often leads to code that is easier to reason about and less prone to bugs.

In summary, the AI-suggested refactoring to a list comprehension is a best-practice improvement that results in cleaner, faster, and more Pythonic code.